



Java Access Modifiers Summary

Access modifiers determine the scope of a class, method, constructor, or field—meaning they control where the element can be accessed or used in your program. They are crucial for implementing **encapsulation** (hiding internal implementation details).

The Four Access Levels

Java has four types of access modifiers, which dictate visibility across classes, packages, and inheritance hierarchies:

Modifier	Access Level	Same Class	Same Package	Subclass (outside package)	World (anywhere)
private	Lowest	✓ Yes	✗ No	✗ No	✗ No
default	Package-Private	✓ Yes	✓ Yes	✗ No	✗ No
protected	Inheritance/Package	✓ Yes	✓ Yes	✓ Yes	✗ No
public	Highest	✓ Yes	✓ Yes	✓ Yes	✓ Yes

Detailed Explanation

1. private (Restrictive)

- **Visibility:** Members declared private are only accessible **within the class where they are defined**.
- **Use Case:** Ideal for internal fields that should not be directly changed by external code. This enforces encapsulation.

2. default (Package-Private)

- **Visibility:** If no modifier is specified, the element is **default**. It is accessible only within the same package.
- **Use Case:** When you want related classes within the same organizational unit (package) to access a field, but nothing outside that unit.

3. protected (Inheritance Focus)

- **Visibility:** Accessible within the **same package** AND in any **subclass**, even if the

subclass is in a different package.

- **Use Case:** Often used for methods or fields that are meant to be customized or extended by child classes.

4. public (Universal)

- **Visibility:** Accessible from **everywhere** (any class, any package).
- **Use Case:** Used for methods and classes that need to be universally callable (e.g., the main method or general utility classes).

Example: Enforcing Encapsulation with private

This example shows how private protects data, requiring access through methods (getters and setters).

```
package com.company.vehicles;
```

```
public class Car {  
    // 1. private field: Can ONLY be accessed/changed inside the Car class  
    private int speed = 0;  
  
    // 2. public method: Allows universal access to the behavior  
    public void accelerate() {  
        speed = speed + 10;  
        System.out.println("Current speed: " + speed);  
    }  
  
    // 3. public getter method: Allows universal, read-only access to the private data  
    public int getSpeed() {  
        return speed;  
    }  
}
```

```
// In a different package, you can't access 'speed' directly:  
// Car myCar = new Car();  
// System.out.println(myCar.speed); // ERROR!  
// System.out.println(myCar.getSpeed()); // OK!
```